architectures. Google adapted these cipher suites for TLS to provide better performance and security on mobile devices that lack AES hardware acceleration.

DEFINITION ***AES-CCM*** (Counter with CBC-MAC) is an authenticated encryption algorithm based solely on the use of an AES encryption circuit for all operations. It uses AES in Counter mode for encryption and decryption, and a Message Authentication Code (MAC) based on AES in CBC mode for authentication. ***ChaCha20-Poly1305*** is a stream cipher and MAC designed by Daniel Bernstein that is very fast and easy to implement in software.

Both of these choices have fewer weaknesses compared to either AES-GCM or the older AES-CBC modes when implemented on constrained devices.[5] If your devices have hardware support for AES, for example in a dedicated secure element chip, then CCM can be an attractive choice. In most other cases, ChaCha20-Poly1305 can be easier to implement securely. Java has support for ChaCha20-Poly1305 cipher suites since Java 12. To force the use of ChaCha20-Poly1305 you can specify a custom `SSLParameters` object and pass it to the `setSSLParameters()` method on the `SSLEngine`. Listing 12.7 shows how to configure the parameters to only allow ChaCha20-Poly1305-based cipher suites. Open the DtlsClient.java file and add the new method to the class.

TIP If you need to support servers or clients running older versions of DTLS you should add the TLS_EMPTY_RENEGOTIATION_INFO_SCSV marker